HYBITS ENTERPRISES

Hotspot Billing Module

Development stages for Claude Code & progress monitoring guide

Prepared for	Hillary
Date	16 October 2026
Version	1.0
Systems	ISP Billing app (tenant DB) and Customer Management System (master DB)
Stack	Laravel + MySQL, MikroTik RouterOS API over VPN, M-Pesa Daraja
Estimated duration	About 60 working days (15 stages, including the switch to Git deployment)

This document breaks the Hotspot Management Module into stages that Claude Code builds one at a time. Each stage has a ready-to-paste prompt, the files and features it must produce, an acceptance checklist you tick yourself, and a sign-off line. Print it and keep it next to you while the work runs.

1. How to run this project with Claude Code

The rule is simple: **one stage per Claude Code session, and nothing moves forward until you have ticked that stage's checklist.** This keeps every change small enough for you to review.

Workflow for every stage

- **Start a fresh session** in the right repo (billing app, CMS, or both) on the **feature/hotspot** branch.
- **Paste the stage prompt** and ask Claude Code to start in plan mode. Read the plan; correct it before any code is written.
- **Let it build**, then ask it to run the tests and show you what changed (files and migrations).
- **Check it yourself** on staging using the acceptance checklist. Tick each box only after you have seen it work.
- **Commit and tag locally** (for example **hotspot-stage-05**), write the tag on the sign-off line, and update the progress tracker. Git is used on your development machine only until Stage 14; the live servers keep their current manual upload until then.
- **Run the baseline regression checklist** from Stage 0 to confirm the existing billing system and CMS still behave exactly as before.
- **If something fails,** stay in the same stage: describe what you saw and let Claude Code fix it. Don't start the next stage with known bugs.

Golden rules (put these in CLAUDE.md in both repos)

- Never run migrations, seeders or router commands against production. Use staging and the test router only.
- Never change or drop existing ISP billing tables or columns; only add.
- Respect the do-not-touch list in docs/hotspot/00-impact-and-safety.md. If a change to existing code is unavoidable, stop and explain it before making it.
- Hotspot is OFF by default for every organization; all hotspot routes, jobs and cron entries run only for organizations with hotspot enabled.
- Hotspot jobs use their own queue (hotspot) so they never delay existing billing jobs.
- Re-run the Stage 0 baseline regression checklist at the end of every stage.
- Tenant data stays in tenant DBs; organization billing and M-Pesa credentials stay in the master DB.
- M-Pesa secrets are always encrypted and never logged, echoed or copied to tenant DBs.
- Every router write goes through a queued job with timeout, retry and a per-router lock.
- Every money-related action is idempotent (safe to run twice) and audited.
- Reuse the existing layout, components, RouterOS client, SMS gateway and invoice flow; don't build parallel ones.
- Write tests for each stage and keep docs/hotspot/ up to date.

What to give Claude Code at the start of each session

- This document's stage section (copy the prompt)
- docs/hotspot/00-discovery.md (created in Stage 0)
- Any notes or decisions from earlier stages that changed the plan

2. Confirmed decisions

Area	Decision
Application	Hotspot is a second mode inside the existing ISP billing dashboard, using the same UI template. Users switch with a toggle; only the sidebar and pages change.
Stack	Laravel + MySQL for both the billing app and the CMS.
Tenancy	Existing multitenant design: one master DB (CMS) and one DB per organization. Hotspot operational data lives in tenant DBs.
Payments from end users	Each ISP uses its own M-Pesa Paybill/Till. Credentials are managed in the CMS (master DB) and used by the billing app at payment time.
Billing ISPs for hotspot	The CMS bills each organization for hotspot use. Default model: **percentage of hotspot sales**. A system admin can switch an organization to flat monthly, per active router, or per ticket, and change the rate.
Router connection	RouterOS API over the existing VPN. Remote access also goes through the VPN.
Live operations	Current billing and CMS operations must not be disrupted. Stage 0 studies both systems and produces an impact map, a do-not-touch list and a baseline regression checklist used at every stage.
Deployment	Today: manual upload to the live servers. During development: local git only. After go-live: switch both apps to Git-based deployment (Stage 14).

Where data lives

Master DB (CMS)	Tenant DB (each ISP)
org_hotspot_settings (enabled, billing model, rate) org_mpesa_credentials (encrypted) hotspot_billing_settings (global defaults) org_hotspot_sales_summaries (monthly figures, invoice link) Existing organizations, invoices, payments	hotspot_plans, hotspot_tickets, hotspot_payments hotspot_customers, hotspot_usage_snapshots hotspot_login_pages (branding) routers (+ hotspot columns) Existing ISP clients, PPPoE billing data

How money and data flow

→ Customer on hotspot → branded portal → STK Push to the **ISP's own Paybill** → callback to Hybits → ticket created on the router → customer signed in automatically, SMS with code as fallback.

→ Every month the CMS reads each organization's paid hotspot sales from its tenant DB, applies that organization's billing model, and adds a **Hotspot** line to its normal CMS invoice.

→ Because ISPs collect the money directly, Hybits' share is **invoiced**, not deducted. Sales figures come from M-Pesa callbacks recorded by the system, so they are reliable.

3. Progress tracker

Fill this in as you go. Status: Not started / In progress / Testing / Done.

#	Stage	Est. days	Needs	Status	Started	Done
0	Study the live system & protect current operations	4	—			
1	Database foundation (master + tenant)	3	0			
2	Mode switch & shared UI	2	1			
3	CMS: enable hotspot, M-Pesa credentials, billing model	4	1			
4	Routers: hotspot detection & setup over VPN	4	2			
5	Hotspot plans	3	4			
6	Tickets & provisioning engine	6	5			
7	Captive portal & custom login page	5	5			
8	M-Pesa STK Push & automatic sign-in	6	3, 6, 7			
9	Reports	4	6, 8			
10	Router remote access	4	4			
11	CMS hotspot billing engine	5	3, 8			
12	Testing, load & hardening	5	all			
13	Deployment, documentation & training	2	12			
14	Switch live updates to Git-based deployment	3	13			
	Total	**60**				

Mapping to the client proposal phases: Planning & design = Stage 0–2; Module development = Stages 3–6, 8–11; Custom login page = Stage 7; Testing & optimization = Stage 12; Implementation = Stage 13. Stage 14 (Git deployment) is internal and follows go-live. Stages 7 and 10 can run in parallel with others once their dependencies are done.

4. Stages in detail

Stage 0 — Study the live system & protect current operations

~4 days · needs —

Goal. Before any code is written, Claude Code studies the current ISP billing app and the CMS in depth: how billing, activation/deactivation and organization invoicing work today, and exactly what the hotspot work could touch. The output is a written safety map and a baseline of today's behaviour that every later stage is checked against, so live ISPs are never disrupted.

Prompt for Claude Code

```
This session is READ-ONLY for application code. Do not modify, run migrations, or call
any router or payment API.
Study BOTH repositories: the ISP billing app (tenant side) and the CMS (master side).
Write docs/hotspot/00-discovery.md covering:
1. Tenancy: package or custom code, how the tenant DB connection is resolved per request
and in jobs/cron, how tenant migrations are run, how a new organization is provisioned.
2. Current billing cycle end to end: monthly invoice generation, payment recording,
M-Pesa (if any), reminders, and how clients are activated/deactivated on MikroTik
(which code, which schedule, which router commands).
3. CMS: how organizations are billed, invoices, payments, suspension of organizations,
and how the CMS talks to tenant DBs.
4. Routers: how they are stored, the RouterOS API client, VPN addressing, timeouts,
error handling, and any scripts or scheduler entries that write to routers.
5. Scheduler (cron) entries, queue workers, queue names, SMS gateway, mail, logging.
6. UI: main layout, sidebar, top bar, permissions/roles system, assets build.
7. Server: PHP version, web server, MySQL version, how files are currently deployed
(manual upload), .env handling, storage/symlinks.
Then write docs/hotspot/00-impact-and-safety.md:
A. Impact map: every existing table, model, route, middleware, layout file, cron entry,
job, router script and config the hotspot module will READ or CHANGE, with the
reason. Anything that would be CHANGED must be listed with how it stays
backward-compatible.
B. Do-not-touch list: files, tables and behaviours that must not change at all
(e.g. the monthly billing run, PPPoE activation/deactivation, CMS invoicing logic).
C. Baseline regression checklist: 10-20 key flows of today's system written as
step-by-step manual checks (and automated tests where safe), e.g. generate monthly
invoices, record a payment, auto-deactivate an unpaid client, reactivate on payment,
CMS invoices an organization, create a new organization.
D. Risks and open questions for me to answer before Stage 1.
Finally create CLAUDE.md in each repo with the Golden rules from this plan and a link to
both docs. If a repo is not yet under git, initialise LOCAL git only (no remote), with a
.gitignore that excludes .env, vendor, node_modules and storage files, and make an
initial commit of the code exactly as it is now.
```

Deliverables

- docs/hotspot/00-discovery.md (how the current system works)
- docs/hotspot/00-impact-and-safety.md (impact map, do-not-touch list, baseline regression checklist)
- CLAUDE.md in both repos
- Local git in both repos with a baseline commit tagged **pre-hotspot-baseline**; branch **feature/hotspot**
- Staging copy of both apps with a recent backup of the master DB and at least one tenant DB
- A test MikroTik (physical or CHR) reachable over the VPN

Acceptance checklist (tick after you have seen it work)

- ☐ The local code copy matches what is on the live servers (compare files first, since live has been updated manually)
- ☐ Discovery file correctly describes billing, activation/deactivation and CMS invoicing (you have read and confirmed it)
- ☐ Impact map reviewed: nothing on the do-not-touch list is planned to change
- ☐ Baseline regression checklist run once on staging and all flows pass
- ☐ No production database, router or Paybill is used for development
- ☐ Backups restored successfully on staging at least once
- ☐ Open questions answered before Stage 1

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 1 Database foundation (master + tenant)

~3 days · needs 0

Goal. Create every table the module needs, in the right database, with migrations that run cleanly for all existing tenants and for newly created ones.

Prompt for Claude Code

```
Read docs/hotspot/00-discovery.md and this stage of the plan first. Start in plan mode.
TENANT DB migrations (must run through the existing tenant-migration mechanism):
- hotspot_plans: name, price, upload_speed, download_speed, rate_limit string,
validity_type (paused|elapsed), duration_minutes, data_limit_mb nullable,
shared_users, is_active, sort_order, timestamps, soft deletes.
- hotspot_tickets: code (unique), phone nullable, mac nullable, plan_id, router_id,
payment_id nullable, source (stk|manual), status (pending|active|used|expired|failed),
activated_at, expires_at, uptime_used_seconds, bytes_in, bytes_out, created_by nullable.
- hotspot_payments: phone, amount, plan_id, router_id, mac, ip, checkout_request_id (unique),
merchant_request_id, mpesa_receipt (unique nullable), status (pending|paid|failed|cancelled),
result_code, result_desc, raw_callback json, paid_at.
- hotspot_login_pages: logo_path, primary_color, secondary_color, title, welcome_text,
support_phone.
- hotspot_customers: phone (unique), first_seen_at, last_purchase_at, purchase_count, total_spent.
- hotspot_usage_snapshots: ticket_id, router_id, bytes_in, bytes_out, uptime_seconds, captured_at.
- Add to existing routers table: hotspot_enabled, hotspot_server_name, hotspot_interface,
hotspot_dns_name, hotspot_last_sync_at.
MASTER DB migrations:
- org_hotspot_settings: org_id, hotspot_enabled, enabled_at, billing_model
(percentage|flat_monthly|per_router|per_ticket), rate decimal, minimum_charge nullable,
effective_from date.
- org_mpesa_credentials: org_id, environment (sandbox|production), shortcode_type
(paybill|till), shortcode, party_b, consumer_key, consumer_secret, passkey (all secrets
encrypted with Laravel casts), callback_token, is_verified, last_verified_at.
- hotspot_billing_settings: global default billing_model and rate (default model = percentage).
- org_hotspot_sales_summaries: org_id, period (YYYY-MM), gross_sales, ticket_count,
stk_ticket_count, manual_ticket_count, active_routers, billing_model, rate,
amount_due, invoice_id nullable, locked_at.
Add models, relationships, factories and seeders for demo data. Do not touch existing columns.
```

Deliverables

- Tenant + master migrations, models, factories, seeders
- A short note in the docs folder listing every new table and which DB it lives in

Acceptance checklist (tick after you have seen it work)

- ☐ **php artisan migrate** on master and the tenant-migrate command for all tenants run without errors on staging
- ☐ Rolling back the new migrations leaves existing tables untouched
- ☐ Secrets in org_mpesa_credentials are stored encrypted (check the raw DB value)
- ☐ Creating a brand-new organization also gets the hotspot tables

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 2 Mode switch & shared UI

~2 days · needs 1

Goal. Users switch between ISP Billing and Hotspot inside the same dashboard and template. Only the sidebar and the pages change.

Prompt for Claude Code

```
Add a mode switch to the existing billing dashboard. Keep the current layout and styling.
- Route groups: existing routes stay as they are; new routes under /hotspot with name
prefix hotspot. Mode is determined by the URL, not the session.
- Middleware SetBillingMode shares $billingMode ('isp' or 'hotspot') with all views.
- Middleware EnsureHotspotEnabled blocks /hotspot routes (403 page with a friendly
message) when the organization's hotspot_enabled flag in the master DB is false.
Cache that flag for 5 minutes.
- Sidebar menu defined in config (isp and hotspot keys) and rendered by the layout.
- Toggle in the top bar ("ISP Billing | Hotspot") shown only when hotspot is enabled.
- Hotspot dashboard placeholder page with cards: today's sales, active users,
tickets sold today, routers online (data can be zero for now).
- Add hotspot permissions to the existing roles/permissions system.
```

Deliverables

- Working toggle and hotspot sidebar
- Hotspot dashboard skeleton

Acceptance checklist (tick after you have seen it work)

- ☐ Toggle hidden for an org with hotspot disabled; /hotspot URLs return 403 for it
- ☐ Two browser tabs open on different modes behave correctly
- ☐ ISP billing pages look and work exactly as before (spot-check 5 pages)
- ☐ Mobile layout of the toggle is usable

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 3 — CMS: enable hotspot, M-Pesa credentials, billing model

~4 days · needs 1

Goal. The CMS (system admin) turns hotspot on per organization, manages each ISP's own Paybill/Till credentials, and sets how that organization is billed for hotspot. Percentage of hotspot sales is the default.

Prompt for Claude Code

```
Work in the CMS repository.
1. Global settings page "Hotspot billing defaults": default billing model (percentage)
and default rate. Only system admins can edit.
2. On each organization's page add a "Hotspot" tab:
- Enable/disable hotspot (writes org_hotspot_settings; on enable copy the global
default model and rate).
- Billing model selector: percentage of sales | flat monthly | per active router |
per ticket, plus rate and optional minimum monthly charge.
- A model/rate change takes effect from the next billing period (effective_from);
keep a history table/audit log of who changed what and when.
- M-Pesa credentials form: environment, Paybill or Till, shortcode, PartyB/account,
consumer key, consumer secret, passkey. Secrets are never shown again after saving
(show "••••" and a "replace" button).
- "Test credentials" button: request a Daraja OAuth token and report success/failure.
3. Expose a read-only service the billing app uses to load the org's credentials and
hotspot flag from the master DB. Never copy secrets into tenant DBs.
4. Generate a per-org callback_token used in the M-Pesa callback URL.
```

Deliverables

- CMS hotspot defaults page
- Per-organization Hotspot tab with billing model + M-Pesa credentials
- Audit log of billing changes

Acceptance checklist (tick after you have seen it work)

- ☐ New org enabled for hotspot gets percentage model + default rate automatically
- ☐ Admin can change model to flat / per router / per ticket and it shows effective next period
- ☐ Wrong consumer key shows a clear 'test failed' result; correct sandbox key passes
- ☐ Secrets do not appear in page source, logs, or API responses
- ☐ Only system admins can see or edit this tab

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 4 Routers: hotspot detection & setup over VPN

~4 days · needs 2

Goal. From the dashboard the ISP sees which routers are hotspot-ready, enables or disables hotspot management on them, and the system keeps router profiles in sync.

Prompt for Claude Code

```
Reuse the existing RouterOS API client and VPN addressing found in discovery.
- Service class HotspotRouterService with: detect() (list /ip hotspot servers, profiles,
RouterOS version), enable(router), disable(router), syncProfiles(router), health(router).
- "Routers" page in hotspot mode: list routers with status (online/offline via VPN),
RouterOS version, hotspot detected yes/no, hotspot managed on/off, last sync.
- Enable flow: choose hotspot server/interface, make sure the walled garden contains the
billing domain (and any CDN it needs), set the hotspot login page redirect (Stage 7),
then mark hotspot_enabled.
- Disable flow: stop managing; do NOT delete anything on the router without confirmation.
- All router calls go through queued jobs with timeouts, retries (3, backoff), and a
per-router lock so two jobs never write to the same router at once.
- Log every router command with result in a router_activity log.
```

Deliverables

- Hotspot Routers page
- HotspotRouterService + queued jobs
- Router activity log

Acceptance checklist (tick after you have seen it work)

☐ Test router is detected as hotspot-ready over the VPN

☐ Offline router shows offline within a minute and does not block the page

☐ Enable/disable works and is logged

☐ RouterOS v6 and v7 both tested (if you have both in the field)

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 5 Hotspot plans

~3 days · needs 4

Goal. ISPs create plans with speeds, price, validity type (paused or elapsed), and duration, and the plans become profiles on their routers.

Prompt for Claude Code

```
CRUD for hotspot plans in hotspot mode.
- Fields: name, price, upload and download speed (with unit), validity type, duration
(minutes/hours/days/weeks/months), optional data limit, shared users, active flag, order.
- Explain validity types in the form:
Paused = time counts only while connected (MikroTik limit-uptime).
Elapsed = time runs continuously from activation (enforced by our scheduler).
- On save/update dispatch a job that creates/updates a matching
/ip hotspot user profile on every hotspot-enabled router (rate-limit, shared-users).
- Deactivating a plan hides it from the login page but does not affect sold tickets.
- Plans list shows which routers are in sync.
```

Deliverables

- Plans page
- Profile sync job

Acceptance checklist (tick after you have seen it work)

- ☐ Created plan appears as a user profile on the test router with correct rate-limit
- ☐ Editing speed updates the router profile
- ☐ Inactive plan disappears from the plan list API used by the login page
- ☐ Price/duration validation prevents zero or negative values

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 6 Tickets & provisioning engine

~6 days · needs 5

Goal. The core engine: create tickets (manually or after payment), push them to the router, track usage, and expire them correctly for both paused and elapsed plans.

Prompt for Claude Code

```
1. TicketService::issue(plan, router, phone, mac, source, payment?) -> ticket with unique
short code; dispatch ProvisionTicket job that runs
/ip hotspot user add name=CODE password=CODE profile=PLAN limit-uptime=(paused plans)
limit-bytes-total=(if data limit).
2. Elapsed plans: set expires_at on first activation; scheduler every minute expires due
tickets, removes the router user and kicks the active session.
3. Paused plans: mark used when the router reports limit-uptime reached.
4. Usage sync job every 5 minutes per router: read /ip hotspot active and /ip hotspot user
counters, update tickets, write hotspot_usage_snapshots.
5. "Tickets bought" page: filter by date (default today), status, router, plan, phone;
show code, phone, plan, status, time used/remaining, data used; actions: view,
resend SMS, disconnect, expire now.
6. Manual ticket form (create for any phone number, optional "send SMS").
7. Make provisioning idempotent: re-running the job must not create duplicates.
```

Deliverables

- TicketService, provisioning/expiry/usage jobs, scheduler entries
- Tickets Bought page + manual ticket form

Acceptance checklist (tick after you have seen it work)

- ☐ Manual ticket logs in on the test router with the code
- ☐ Elapsed 10-minute ticket is cut off ~10 minutes after first login even if disconnected meanwhile
- ☐ Paused 10-minute ticket keeps remaining time after disconnecting
- ☐ Router offline during provisioning: job retries and succeeds when it comes back
- ☐ Status counts on the page match the router

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 7 Captive portal & custom login page

~5 days · needs 5

Goal. Customers on the hotspot see the ISP's branded page with plans, and each ISP customises logo and colours from the dashboard.

Prompt for Claude Code

```
1. Hosted portal route /hs/{router_key} (outside the auth middleware, inside walled
garden). It receives mac, ip, link-login-only, link-orig from the router.
2. Generate a minimal router login.html (and alogin.html, status.html) that redirects to
the hosted portal with those variables. Provide a "Download router files" button and,
where possible, upload them via the API.
3. Portal shows: logo, title, welcome text, active plans (price, speed, duration),
phone input, Buy button, "I have a ticket code" login option, support phone.
4. Branding page in hotspot mode: upload logo (size/type limits), choose primary and
secondary colours, title, welcome text, live preview.
5. Page must be very light: no heavy frameworks, total < 150 KB, works on cheap Android
phones and old browsers.
```

Deliverables

- Hosted portal
- Router login files + download/upload
- Branding page with preview

Acceptance checklist (tick after you have seen it work)

☐ Connecting a phone to the test hotspot opens the branded portal

☐ Ticket code login works from the portal

☐ Changing logo/colours shows on the portal without touching the router

☐ Page loads fast on a slow connection (test with a throttled browser)

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 8 M-Pesa STK Push & automatic sign-in

~6 days · needs 3, 6, 7

Goal. Customer picks a plan, enters a phone number, pays via STK Push to the ISP's own Paybill, and is signed in automatically. An SMS with the ticket code is the fallback.

Prompt for Claude Code

```
1. Load the org's Daraja credentials from the master DB (Stage 3 service); cache the
OAuth token until it expires.
2. POST /hs/{router_key}/pay: validate plan + phone (format 2547XXXXXXXX / 2541XXXXXXXX),
rate-limit per phone and per IP, create hotspot_payments row (pending), send STK Push
with callback URL /api/hotspot/mpesa/callback/{org_uuid}/{callback_token}.
3. Callback controller: resolve tenant from org_uuid, verify callback_token, match
CheckoutRequestID, be idempotent (same receipt twice = one ticket), store raw payload,
respond to Safaricom immediately and do the rest in a queued job.
4. On success: issue ticket (Stage 6), provision router, update hotspot_customers,
send SMS with the ticket code.
5. GET /hs/status/{payment_id}: cheap status endpoint (cache, no heavy queries). The
portal polls every 4 seconds, up to 2 minutes, then shows "Check SMS / enter code".
6. When paid: the portal auto-submits username/password to link-login-only.
7. If no callback within 2 minutes, query Daraja STK status and reconcile.
8. Payments page in hotspot mode: list, filters, totals, export CSV.
```

Deliverables

- STK Push flow end-to-end
- Callback + reconciliation

- Payments page

Acceptance checklist (tick after you have seen it work)

- ☐ Sandbox payment creates a ticket and logs the phone in without typing a code
- ☐ Cancelled STK shows a clear message; no ticket created
- ☐ Duplicate callback does not create two tickets
- ☐ Wrong callback_token is rejected and logged
- ☐ Callback responds in under 1 second (heavy work is queued)
- ☐ Real KES 1–10 test on production Paybill succeeds before go-live

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 9 Reports

~4 days · needs 6, 8

Goal. Insight for the ISP: retention, daily purchases, and data usage.

Prompt for Claude Code

```
Reports section in hotspot mode, each with date range filter and CSV/PDF export:
1. Customer retention: new vs returning buyers per period, % returning after first
purchase, top repeat customers (by phone).
2. Daily ticket purchases: all purchases on a chosen day with time, phone, plan, amount,
router, source (STK/manual).
3. Data usage: per-user (phone/ticket) and overall per router, from usage snapshots.
4. Sales summary: revenue by day, plan, router.
Use aggregated queries with indexes; check query plans on a tenant with 100k tickets
(seed it).
```

Deliverables

- Four reports with exports
- Index migration if needed

Acceptance checklist (tick after you have seen it work)

- ☐ Totals match the Payments page for the same period
- ☐ Reports load in under 3 seconds on the 100k-ticket seed
- ☐ Manual tickets clearly separated from paid tickets

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 10 Router remote access

~4 days · needs 4

Goal. Administrators manage routers from anywhere through the existing VPN.

Prompt for Claude Code

```
Router detail page (hotspot mode, admin permission only):
- Live status: uptime, CPU, memory, RouterOS version, VPN status.
- Connected devices: hotspot active users, DHCP leases.
- Recent logs (/log print, last 100) with refresh.
- Actions: reboot (with confirmation), disconnect user, re-sync profiles and login files.
- Winbox/WebFig access through the VPN (document the approach; no router ports exposed
to the public internet).
- Firmware: show current and available version only. Upgrades require typing the router
name to confirm and are off by default.
Every action is permission-checked and written to the router activity log.
```

Deliverables

- Router detail page
- Documented remote-access method

Acceptance checklist (tick after you have seen it work)

☐ Reboot of the test router works and comes back online on the dashboard

☐ A user without the permission cannot see actions

☐ No router management port is open to the internet (scan it)

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 11 CMS hotspot billing engine

~5 days · needs 3, 8

Goal. The CMS calculates what each organization owes Hybits for hotspot each month and adds it to the existing CMS invoicing. Percentage of sales is the default model.

Prompt for Claude Code

```
Work across both repos.
1. Monthly job (1st of the month, and an on-demand "recalculate" for admins) loops through
orgs with hotspot enabled, connects to each tenant DB, and computes for the previous
month: gross_sales (paid STK payments only), ticket_count, stk_ticket_count,
manual_ticket_count, active_routers (hotspot-enabled routers seen online in the month).
2. Use the billing model and rate effective for that period:
percentage -> gross_sales x rate%
flat_monthly -> rate
per_router -> active_routers x rate
per_ticket -> stk_ticket_count x rate
then apply minimum_charge if set.
3. Save to org_hotspot_sales_summaries, then create the charge through the CMS's
EXISTING invoice flow (as a separate "Hotspot" line item). Lock the summary once
invoiced; recalculation of a locked period requires admin confirmation.
4. CMS page "Hotspot billing": per org, per month: sales, model, rate, amount due,
invoice link, status. Org-side view in the billing app shows the same statement.
5. Mid-month live view in the CMS: month-to-date sales and estimated charge.
6. Unit tests for every billing model, including zero sales and mid-period rate changes.
```

Deliverables

- Monthly calculation job + manual recalculate
- Hotspot line item on CMS invoices
- CMS and org-side hotspot billing statements

- Tests for all four models

Acceptance checklist (tick after you have seen it work)

☐ Percentage example: KES 120,000 sales at 5% gives KES 6,000 on the invoice
☐ Rate change on the 15th applies to next month, not the current one
☐ An org with hotspot disabled is not billed
☐ Org's own statement matches the CMS figure
☐ Running the job twice for a month does not double-bill

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 12 Testing, load & hardening

~5 days · needs all

Goal. Prove the module is correct and the server can carry it before real customers touch it.

Prompt for Claude Code

```
1. Feature tests for: mode switch, plan sync, ticket lifecycle (paused & elapsed),
STK success/cancel/duplicate/timeout, billing models.
2. Load test the portal, /hs/status and the callback endpoint (e.g. k6 or ab) with
200 concurrent portal users; report response times and errors.
3. Review server config for the extra load: PHP-FPM/Apache MPM, worker counts, MySQL
indexes, queue workers under supervisor, scheduler cron.
4. Security review: callback auth, rate limits, CSRF exceptions only where needed,
permission checks, encrypted secrets, no secrets in logs.
5. Write docs/hotspot/test-report.md with results and any fixes made.
```

Deliverables

- Test suite passing
- Load test results
- docs/hotspot/test-report.md

Acceptance checklist (tick after you have seen it work)

☐ All tests green
☐ Portal + status endpoint stay fast at 200 concurrent users without Apache worker exhaustion
☐ Queue workers restart automatically after a crash/reboot
☐ Security checklist items all ticked in the report

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 13 Deployment, documentation & training

~2 days · needs 12

Goal. Go live with a pilot ISP, then roll out.

Prompt for Claude Code

```
1. Write docs/hotspot/deployment.md for the CURRENT manual process: exact list of new
and changed files per repo (from git diff pre-hotspot-baseline..feature/hotspot
--name-only), migration order (master first, then tenants), env variables,
queue/scheduler changes, a pre-upload backup step and rollback steps (restore the
backed-up files and DBs).
2. Write the ISP user guide (plans, tickets, routers, branding, payments, reports) and the
CMS admin guide (enable hotspot, credentials, billing models, invoices).
3. Write a router onboarding checklist (VPN, hotspot server, walled garden, login files).
4. Prepare a pilot checklist for one ISP with 1–2 routers for one week.
```

Deliverables

- Deployment + rollback guide
- ISP user guide and CMS admin guide
- Router onboarding checklist
- Pilot completed with one ISP

Acceptance checklist (tick after you have seen it work)

- ☐ Backups of files and DBs taken right before uploading
- ☐ Baseline regression checklist from Stage 0 passes on live after upload
- ☐ Pilot ISP sold real tickets for a week with no manual fixes
- ☐ First hotspot invoice generated correctly for the pilot
- ☐ Training session held with the pilot ISP

Stage approved by: _____ Date: _____ Commit / tag: _____

Stage 14 Switch live updates to Git-based deployment

~3 days · needs 13

Goal. After the hotspot module is live and stable, replace manual file uploads with a repeatable Git-based deployment for both the billing app and the CMS, so future updates are traceable and can be rolled back in minutes.

Prompt for Claude Code

```
Plan first and do not run anything on the live servers yourself; give me commands to run.
1. Reconcile: produce a script that lists differences between the live server files and
the repo (excluding .env, vendor, storage, uploads). Any live-only change must be
brought into the repo before we switch.
2. Create a private remote repository (GitHub or GitLab) for each app; push main and
tags. Protect main; feature work happens on branches and is merged when tested.
3. Server setup: deploy key (read-only) per server, clone into a new releases folder,
keep .env, storage and user uploads in a shared folder linked into each release.
4. Write deploy.sh: fetch the chosen tag, composer install --no-dev, migrate master then
all tenants, config/route/view cache, restart queue workers, switch the "current"
symlink only after all steps succeed (zero-downtime release folders).
5. Write rollback.sh: point "current" back to the previous release; document when a DB
rollback is needed and how.
6. Optional: a GitHub Actions/GitLab CI job that runs tests on every push.
7. Update docs/deployment.md and CLAUDE.md: manual uploads are no longer used.
```

Deliverables

- Reconciliation report (live vs repo)

- Private remote repos for billing app and CMS
- deploy.sh and rollback.sh with release folders
- Updated deployment docs and CLAUDE.md

Acceptance checklist (tick after you have seen it work)

☐ Reconciliation shows no unexplained differences between live and repo
☐ A trivial change (e.g. footer text) deployed to staging with deploy.sh
☐ rollback.sh restores the previous release on staging in under 2 minutes
☐ First real live deployment done with the script during a quiet hour, baseline checks pass
☐ .env and uploads survive a deployment untouched

Stage approved by: _____ Date: _____ Commit / tag: _____

5. End-to-end scenarios before go-live

Run all of these on staging with the test router and a real phone. Then repeat 1, 2 and 7 on the pilot ISP.

- ☐ 1. New customer buys a 1-hour paused plan by STK and is online without typing a code.
- ☐ 2. Same customer disconnects after 20 minutes, reconnects later with the SMS code and has ~40 minutes left.
- ☐ 3. Customer buys a 1-day elapsed plan; it ends 24 hours after first login regardless of usage.
- ☐ 4. Customer cancels the STK prompt; portal shows a clear message and no ticket exists.
- ☐ 5. M-Pesa callback arrives late (simulate); reconciliation still issues the ticket once.
- ☐ 6. Router loses VPN for 10 minutes; tickets bought meanwhile are provisioned when it returns.
- ☐ 7. ISP changes logo and colours; next portal visit shows them.
- ☐ 8. Admin switches between ISP Billing and Hotspot; ISP billing still works exactly as before.
- ☐ 9. Month-end: CMS computes the hotspot charge with percentage model and it appears on the invoice.
- ☐ 10. Admin changes an org to per-router billing; next month's invoice uses the new model.
- ☐ 11. Organization with hotspot disabled cannot open /hotspot pages and is not billed.
- ☐ 12. Load test: 200 concurrent portal users, no server errors or Apache worker exhaustion.
- ☐ 13. Full Stage 0 baseline regression checklist passes: monthly billing, PPPoE activation/deactivation and CMS invoicing unchanged.

6. Risks and open decisions

Item	Why it matters	Suggested handling
Server capacity	Portal polling and M-Pesa callbacks add constant load; the current server has had Apache worker exhaustion.	Upgrade or tune before Stage 12; keep status endpoint cache-only; queue all heavy work.
Manual tickets under % billing	Manual tickets bring no recorded payment, so an ISP could sell cash tickets outside the percentage.	Decide: bill manual tickets per ticket, cap them, or accept. Reports show them separately.
Non-payment by an ISP	ISPs collect money directly; Hybits must invoice and collect its share.	Decide whether unpaid hotspot invoices suspend hotspot mode (reuse existing CMS suspension rules if any).
MAC randomisation	Phones change MAC per network, so MAC is unreliable for recognising customers.	Use phone number as the customer key; MAC only for the current session.
Remote firmware upgrades	A failed upgrade on a remote site needs a physical visit.	Show version only by default; upgrades behind strong confirmation, or remove from scope.
Paybill API access	Each ISP must have Daraja (Go-Live) API access for its own shortcode.	Add to the ISP onboarding checklist; use 'Test credentials' in the CMS before enabling.

Appendix: CMS hotspot billing formulas

Rates shown are examples only; actual defaults are set by the system admin in the CMS.

Model	Monthly charge	Example
Percentage of sales (default)	Paid STK sales × rate %	KES 120,000 × 5% = KES 6,000
Flat monthly	Fixed rate	KES 2,500
Per active router	Hotspot routers online in the month × rate	4 × KES 500 = KES 2,000
Per ticket	Paid tickets × rate	3,000 × KES 2 = KES 6,000
Minimum charge (optional)	Charge = max(calculated, minimum)	max(800, 1,000) = KES 1,000